CS695 Project Report

22b3913

Abstract

This report details the work completed for the CS695 project. It covers two major topics: hardware virtualization using KVM in Part 1, and containerization from scratch, including namespaces, cgroups, and service orchestration in Part 2. The report provides a comprehensive overview of the design, implementation, and functionality of the solutions developed for each part of the project.

Contents

1 Part 1: Hardware Virtualization with KVM

1.1 Introduction

This part of the project focuses on understanding and utilizing the Kernel-based Virtual Machine (KVM) API to create a simple hypervisor. The goal is to create, run, and manage a guest virtual machine that executes a simple payload. This part is divided into two main sub-parts. The first deals with setting up a basic virtual machine and running a 16-bit guest. The second extends this by exploring I/O emulation, VCPU scheduling, and running more complex guests.

1.2 Sub-Part 1.1: A Simple Hypervisor (`simple-kvm.c`)

The first sub-part involved writing a C program, `simple-kvm.c`, that acts as a hypervisor. This program uses the KVM API to perform the fundamental steps of creating and managing a VM.

1. **KVM Initialization**: The program starts by opening the KVM device file `/dev/kvm` to get a file descriptor, which is the entry point to the KVM API. It checks the API version for compatibility.
2. **VM Creation**: A new virtual machine is created using the `KVM_CREATE_VM` ioctl. This returns a file descriptor for the newly created VM, which is used for subsequent VM-level operations.
3. **Guest Memory Allocation**: A region of memory is allocated on the host using `mmap` to serve as the guest's physical memory. This memory is then registered with the VM using the `KVM_SET_USER_MEMORY_REGION` ioctl, mapping a guest physical address range to a host virtual address range.
4. **VCPU Creation**: A virtual CPU (VCPU) is created for the VM using the `KVM_CREATE_VCPU` ioctl. A VCPU is represented as a thread in the host and has its own file descriptor.
5. **Loading Guest Code**: The guest code, a simple 16-bit assembly program (`guest16.s`), is loaded directly into the allocated guest memory.
6. **VCPU State Initialization**: The initial state of the VCPU's registers is configured using `KVM_SET_SREGS` and `KVM_SET_REGS`. For the 16-bit real mode guest, this involves setting up segment registers (CS, DS, etc.) and the instruction pointer (RIP) to point to the beginning of the loaded guest code (0x0).
7. **VCPU Execution**: The VCPU is executed by calling the `KVM_RUN` ioctl in a loop. This call is blocking and only returns when a "VM exit" occurs.

1.2.1 The Guest Code (`guest16.s`)

The initial guest is a minimal 16-bit assembly program. Its purpose is to perform a simple operation and then halt.

Listing 1: 16-bit guest code snippet from `guest16.s`

```
.code16
guest16:
        movw $42, %ax
        movw %ax, 0x400
        hlt
```

This code moves the value 42 into the `ax` register, stores it at memory address `0x400`, and then executes the `hlt` instruction. The `hlt` instruction causes a `KVM_EXIT_HLT` VM exit, which the hypervisor uses as a signal to stop the VM and verify the result. The hypervisor checks that register `rax` and the memory at `0x400` both contain the value 42.

1.2.2 Handling VM Exits

The hypervisor's main loop handles VM exits. The `kvm_run` structure provides the exit reason. In the extended version of `simple-kvm.c`, we handle `KVM_EXIT_IO` for basic device emulation. When the guest executes an I/O instruction (like `in` or `out`), it traps to the hypervisor.

Listing 2: VCPU execution loop from `simple-kvm.c`

```
// The main VCPU execution loop
for (;;) {
    if (ioctl(vcpu->vcpu_fd, KVM_RUN, 0) < 0) {
        perror("KVM_RUN");
        exit(1);
    }

    switch (vcpu->kvm_run->exit_reason) {
    case KVM_EXIT_HLT:
        goto check;

    case KVM_EXIT_IO:
        if (vcpu->kvm_run->io.direction == KVM_EXIT_IO_OUT && vcpu->kvm_run->io.port ==
            char *p = (char *)vcpu->kvm_run;
            fwrite(p + vcpu->kvm_run->io.data_offset,
                   vcpu->kvm_run->io.size, 1, stdout);
            fflush(stdout);
        }
        continue;
    // ... other exit reasons
    }
}
```

The code emulates a simple serial port at port `0xE9`. When the guest writes to this port, the hypervisor intercepts the call and prints the character to the host's standard output. This demonstrates the fundamental principle of I/O virtualization where the hypervisor traps and emulates hardware access.

1.3 Sub-Part 1.2: Advanced Emulation and Scheduling

1.3.1 VCPU Scheduling (`emu.c`)

The `emu.c` program was developed to study VCPU scheduling. It creates two independent VMs, each with one VCPU, and runs them concurrently using separate host threads. Each VM runs a simple guest that executes an infinite loop with I/O operations.

Listing 3: Concurrent VM execution in `emu.c`

```
void kvm_run_vm(struct vm *vm1, struct vm *vm2)
{
    pthread_create(&(vm1->vcpus->vcpu_thread), NULL, vm1->vcpus->vcpu_thread_func, vm1);
    pthread_create(&(vm2->vcpus->vcpu_thread), NULL, vm2->vcpus->vcpu_thread_func, vm2);
    pthread_join(vm1->vcpus->vcpu_thread, NULL);
    pthread_join(vm2->vcpus->vcpu_thread, NULL);
}
```

By observing the output and using host tools like `top`, we can see that the two VCPUs are scheduled by the host OS kernel just like any other user-space threads. Their execution is interleaved, demonstrating the host scheduler's role in managing VCPU time. The files `sched1.txt` and `sched2.txt` contain the logs of this interleaved execution.

1.3.2 Running C Code in the Guest (`guest3a.c`)

To demonstrate running more complex code, we compiled a C program to run as a guest. This required setting up a protected mode environment with a stack. The C code performs a simple calculation in a loop and uses a custom hypercall to communicate with the hypervisor.

Listing 4: C guest code from `guest3a.c`

```
static void outl(uint16_t port, uint32_t value)
{
        asm("outl %0,%1" : : "a"(value), "Nd"(port) : "memory");
```

```
}

void HC_produced(int *str)
{
        uint32_t ptr = (uint32_t)((uintptr_t)str);
        outl(0x29, ptr);
}

void _start(void)
{
        int arr[5] = {0};
        for(int i = 0;;i++){
                if(i%5 == 0 && i>0){
                        HC_produced(arr);
                }
                arr[i%5] = i;
        }
}
```

The `HC_produced` function is a hypercall implemented using an `outl` instruction to a specific I/O port (`0x29`). The hypervisor intercepts this `KVM_EXIT_IO`, translates the guest virtual address of the array to a host virtual address, and can then read the contents of the array. This demonstrates a mechanism for structured data exchange between the guest and the hypervisor, which is essential for implementing more complex device models and hypervisor services. The hypervisor's handler for this uses `KVM_TRANSLATE` to safely resolve the guest pointer.

2 Part 2: Containerization from Scratch

2.1 Introduction

This part of the project provides a deep dive into the technologies that underpin modern container systems like Docker. We built a container runtime from the ground up, starting with Linux namespaces and cgroups, and culminating in a service orchestration system. This hands-on approach reveals the core concepts of isolation, resource management, and image layering.

2.2 Task 2.1: Namespaces (`namespace_prog.c`)

The foundation of containerization is kernel namespaces, which allow for the isolation of system resources. We wrote a C program, `namespace_prog.c`, to create new processes in isolated namespaces.

The program uses the `clone()` system call with flags like `CLONE_NEWUTS` (for hostname), `CLONE_NEWPID` (for process IDs), and `CLONE_NEWNS` (for mount points).

- A child process is created with its own UTS and PID namespaces. Inside this child, `getpid()` returns 1, and it can have a different hostname from the parent.
- The parent process can then use `setns()` to enter the namespaces created by its child, allowing subsequent processes forked by the parent to share the same isolated environment.

This task demonstrates how namespaces provide separate views of the system to different processes.

Listing 5: Cloning a process with new namespaces in `namespace_prog.c`

```
child_pid = clone(child_function, child_stack + CHILD_STACK_SIZE,
                  CLONE_NEWUTS | CLONE_NEWPID | SIGCHLD, pipefd);
```

2.3 Task 2.2: Building a Simple Container (`container_prog.c`)

Building on Task 2.1, we created a simple container runtime. This involved:

- A C program (`container_prog.c`) that is executed inside the container to demonstrate its isolation.
- A shell script (`simple_container.sh`) that uses `unshare` and `chroot` to launch the container.
- A dedicated root filesystem for the container (`container_root/`).

The script sets up the namespaces and then changes the root directory to `container_root/`, effectively trapping the containerized process within its own filesystem. We also introduced cgroups to limit the CPU and memory resources the container can use, preventing it from monopolizing host resources. This task combines namespace isolation with filesystem and resource isolation.

2.4 Task 2.3: "Conductor" - A Container Build Tool (`conductor.sh`)

To automate building container images, we created a tool called "Conductor". It mimics the functionality of Docker's build process.

- A `Conductorfile` specifies the build steps (e.g., `FROM`, `COPY`, `RUN`).
- The `conductor.sh` script parses this file and builds a layered image.

A key innovation in this task is the use of the OverlayFS file system to create layered images.

- The `FROM` instruction specifies a base filesystem, which is created using `debootstrap`.
- Each `COPY` or `RUN` instruction creates a new layer on top of the previous ones. A `RUN` command, for example, mounts an overlay with the existing layers as 'lowerdir' and a new empty directory as 'upperdir'. The command is executed within this merged view, and any changes are written to the 'upperdir', which becomes the new layer.
- The final image is simply a stack of these layer directories, which are combined at runtime using OverlayFS. This is highly efficient for storage and distribution.

Listing 6: Mounting an overlay filesystem in `conductor.sh`

```
mount -t overlay overlay -o lowerdir="$parent_layers",upperdir="$CACHEDIR/layers/$layer_
chroot "$temp_mount" /bin/sh -c "$command"
umount "$temp_mount"
```

2.5 Task 2.4: Service Orchestration (`service-orchestrator.sh`)

The final task was to build a service orchestrator to manage a multi-service application. We defined two services:

1. **Counter Service**: A web service written in C (`counter-service.c`) using the `civetweb` library. It exposes HTTP endpoints to manage a simple counter. This service runs in a container built by Conductor.
2. **External Service**: A Python Flask application (`app.py`) that acts as a web-based client for the counter service. It provides a user-friendly interface.

The `service-orchestrator.sh` script automates the deployment:

- It builds the images for both services using their respective `Conductorfile`s (`csfile` and `esfile`).
- It runs two containers, `cs-cont` and `es-cont`, in the background.
- It sets up networking using `veth` pairs and network namespaces. The script creates a virtual ethernet link between each container and the host.
- It configures `iptables` rules to:
  - Allow the containers to access the internet (NAT).
  - Enable peer-to-peer communication between the two containers.
  - Expose the external service's port (8080) on the host's port 3000 (DNAT).
- It retrieves the internal IP address of the counter service container and passes it to the external service container as an environment variable or command-line argument.
- Finally, it launches the services inside their respective containers using `conductor.sh exec`.

This task provides a practical demonstration of microservices architecture, container networking, and the automation required to manage a complete application stack.

Listing 7: External service client code in `app.py`

```
from flask import Flask
import requests
import sys

COUNTER_SERVICE_URL = 'http://192.168.2.2:8080' if len(sys.argv) == 1 else sys.argv[1]

app = Flask(__name__)

@app.route('/')
def hello_world():
    res = requests.get(COUNTER_SERVICE_URL + '/get_and_increment_counter')
    return 'Hello CS695 Explorers! You have sent request to this container ' + res.text

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

The final result is a fully containerized, two-tier web application, orchestrated from scratch.

3 Conclusion

This project provided a comprehensive, hands-on exploration of two fundamental technologies in modern systems: hardware virtualization and containerization.

In Part 1, we built a hypervisor from scratch using the KVM API. This journey from creating a simple VM to run 16-bit code, to handling I/O exits, and finally to managing concurrent VCPUs and running C code, offered deep insights into the mechanics of virtualization. It highlighted the division of labor between the guest, the hypervisor, and the KVM kernel module, and provided practical experience in emulating hardware and managing the VM lifecycle.

In Part 2, we deconstructed the magic of containers. By implementing namespaces, cgroups, a layered image builder with OverlayFS, and a multi-service orchestrator, we recreated the core components of a system like Docker. This exercise demystified how containers achieve isolation, manage resources, and enable complex microservice architectures. The final orchestration of a C-based backend service and a Python-based frontend service showcased the power and flexibility of container-based deployments.

Overall, this project provided a robust understanding of how virtual machines and containers work under the hood, bridging the gap between theoretical concepts and practical implementation.